

Author: Philopatair Mallik
Date: 05/05/2026
purpose: Project 1 - Monopoly Game Simulation

//system libraries
#include //Input/Output library
#include //Random Number Library
#include //Time Library
#include //Format Library
#include //File I/O Library

//User libraries
#include "Cities.h"
#include "Players.h"
#include "Board.h"

//Global Constants - Physics/Chemistry/Conversions/Array Dimensions
int const MAX_BOARD_SPACES = 40; //Total number of spaces on the board
int const MAX_PLAYERS = 4; //Maximum number of players
int const MAX_CITIES = 22+4+2; //22 properties + 4 railroads + 2 utilities
int const MAX_CHANCE_CARDS = 16; //Number of Chance cards
int const MAX_COMMUNITY_CHEST_CARDS = 16; //Number of Community Chest cards
int const BOARD_SIZE = 40; //Total number of spaces on the board
int const STARTINGBALANCE = 1500; //Starting balance for each players
int const SALARY = 200; //Amount collected when passing Go

//Function Prototype
void initialization(string* filesNames, City* cities, Board* board, int* chanceCards, int* communityChestCards);
City readCityFromFile(fstream &inFile, int id);
Board readBoardFromFile(fstream &inFile, int id);
void deallocateMemory(Player* players, City* cities, Board* board, int* chanceCards, int* communityChestCards);
void deallocateMemory(int* array);
void deallocateMemory(string* array);
int initializePlayers(Player* players, int nOfPlyrs);
void getOrderOfPlay(Player* players, int nOfPlyrs);
void sortParallelArray(int *arr, Player* players, int size);
int binarySearch(int *array, int value);
void sortArray(int *array, int size);
void randomizeArray(int *array, int size);
void rollDice(int* diceRoll)
void mainGameLoop(Player* players, Board* board, City* cities, int* chanceCards, int* communityChestCards, int* housesAvailable, int* hotelsAvailable, int nOfPlyrs);
int showMenu(int menuTemplateIndex);
void handleMenuChoice(int choice, int menuTemplateIndex, int playerIndex, Player* players, Board* board, City* cities, int* chanceCards, int* communityChestCards, int* housesAvailable, int* hotelsAvailable, int nOfPlyrs);
void movePlayer(int* doublecount, bool* doubleRoll, int playerIndex, Player* players, Board* board, City* cities, int* chanceCards, int* communityChestCards, int* housesAvailable, int* hotelsAvailable, int nOfPlyrs);
void locationAction(int playerIndex, int rollTotal, Player* players, Board* board, City* cities, int* chanceCards, int* communityChestCards);
void payRentOrTax(int playerIndex, int rollTotal, Player* player, Board* board, City* cities);
void buyProperty(int playerIndex, Player* players, Board* board, City* cities);
void drawChanceCard(int playerIndex, Player* players, int* chanceCards, Board* board, City* cities);
void drawCommunityChestCard(int playerIndex, Player* players, int* communityChestCards, Board* board, City* cities);
void goToJail(int playerIndex, Player* players);
void jailTurn(int playerIndex, Player* players, Board* board, City* cities);
void moneyTransaction(int fromPlayerIndex, int toPlayerIndex, Player* players, int amount);
void moneyTransaction(int playerIndex, Player* players, int amount);
void handleLowBalance(int playerIndex, int amount, Player* players, Board* board, City* cities);
void upgradeProperty(int playerIndex, int* housesAvailable, int* hotelsAvailable, Player* players, Board* board, City* cities);
void mortgageProperty(int playerIndex, Player* players, Board* board, City* cities);
void unmortgageProperty(int playerIndex, Player* players, Board* board, City* cities);
void sellProperty(int playerIndex, Player* players, Board* board, City* cities);
void tradeProperty(int playerIndex, Player* players, Board* board, City* cities);
int declareBankruptcy(int playerIndex, int* housesAvailable, int* hotelsAvailable, Player* players, Board* board, City* cities);
void calculateGroupOwned(int propertyID, int playerIndex, Player* players, City* cities);
void listPlayerProperties(int playerIndex, Player* players, City* cities);
void displayPropertyOneLiner(int propertyID, City* cities);



